

Rockborne

Data Pre-processing

Preparing raw data for modelling

Objectives



Missing values

Impute numeric and categorical gaps.



Scaling

Normalization and standardization.



Feature engineering

Build more predictive features.



Encoding

Turn categories into numbers.



Imbalanced data

Rebalance a rare target.



Leakage-safe pipeline

Split, then chain steps safely.

Why This Matters for Risk Modelling



Missing values

Application data has gaps; most models cannot train on NaN.



Scaling

KNN, SVM and regularised models are distorted by mixed scales.



Feature engineering

Instalment-to-income can beat either raw column at predicting default.



Encoding

Loan purpose has no natural order; models still need numbers.



Class imbalance

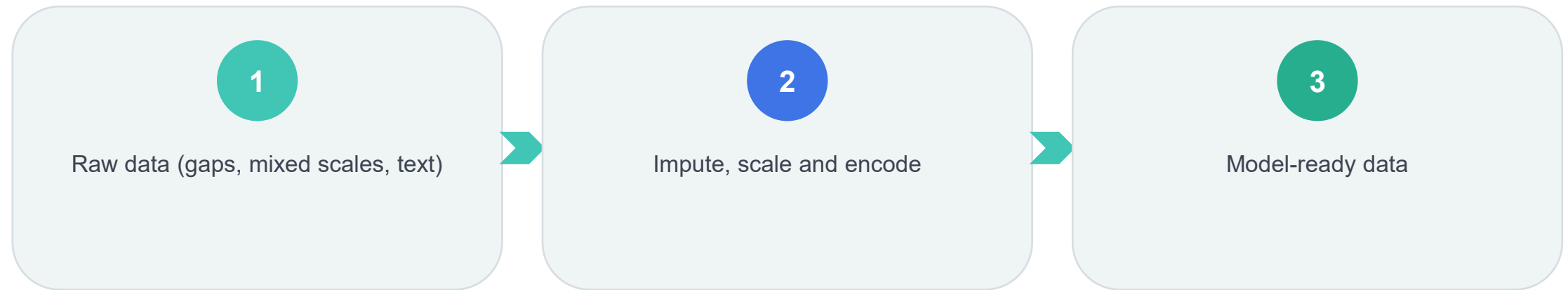
Defaults are rare; a model can score everyone 'good' and look accurate.



No leakage

If preprocessing sees the test set, results are optimistic and fail live.

What is Data Pre-processing?

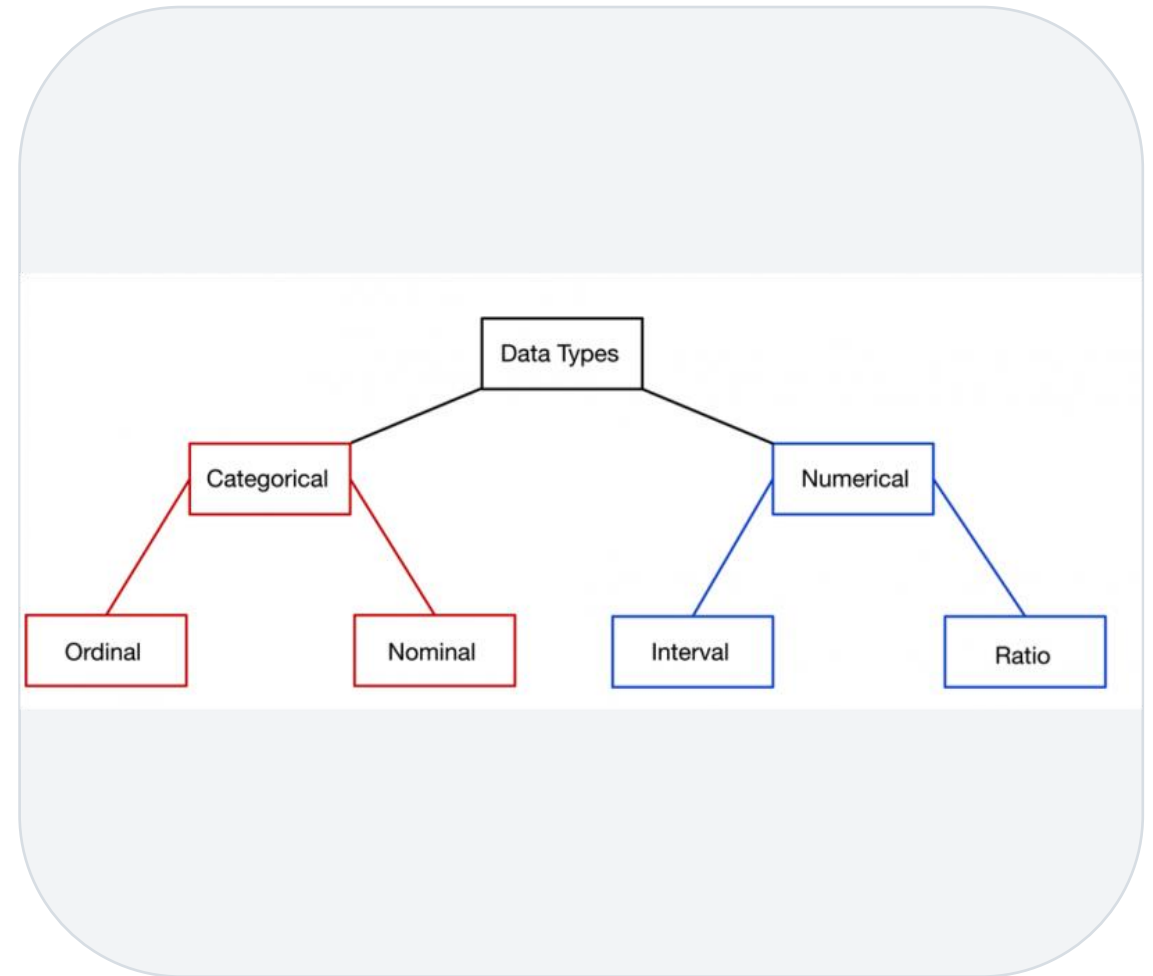


Pre-processing transforms raw data so a model can use it. Get it wrong and there is no error - just a model that looks fine in testing and misbehaves in production.

Types of Data in Statistics

- Nominal: categories with no order (colour, city).
- Ordinal: ordered categories (low, medium, high).
- Interval: numeric, no true zero (temperature).
- Ratio: numeric with a true zero (income, age).

Source: [Built In - Data types in statistics](#)



Scikit-learn: the Preprocessing Toolkit



Imputation

SimpleImputer fills missing values.



Scaling

MinMaxScaler and StandardScaler.



Encoding

OneHotEncoder / get_dummies, LabelEncoder.



Pipeline

Chain steps with Pipeline + ColumnTransformer.

Source: [scikit-learn - Preprocessing data](#)

Data Pre-processing



Cleaning and Scaling Data

Handling Missing Data

Imputation fills gaps so the data is complete enough to model.

Numeric columns

- Most models cannot train with NaN.
- Impute with the mean (SimpleImputer).
- Use the median for skewed money columns.

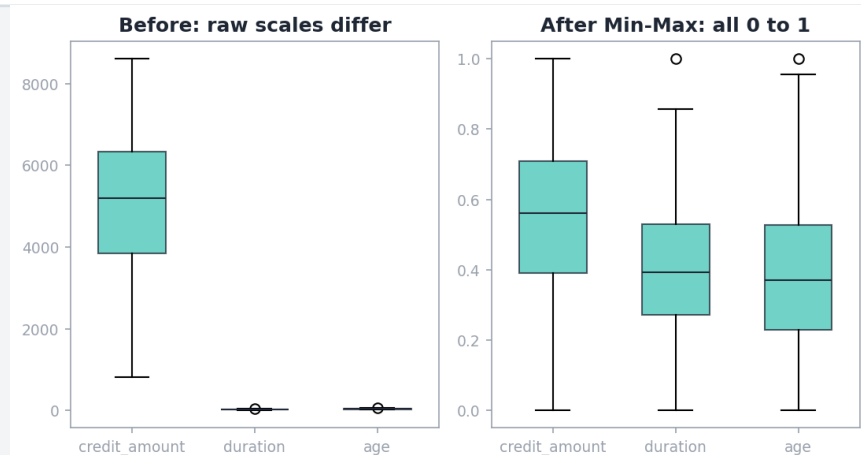
Categorical columns

- A category has no mean.
- Fill blanks with the most frequent value (the mode).
- SimpleImputer(strategy='most_frequent').

Numeric Data: Normalization (Min-Max)

- Rescales each feature to a fixed range, usually 0 to 1.
- Maps the smallest value to 0 and the largest to 1.
- Keeps the shape of the distribution.
- Sensitive to outliers, which stretch the range.

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$



Source: [Machine Learning Mastery - Rescaling data](#)

Normalization: Python vs SQL

Same word, two very different meanings - do not confuse them.

Python (ML)

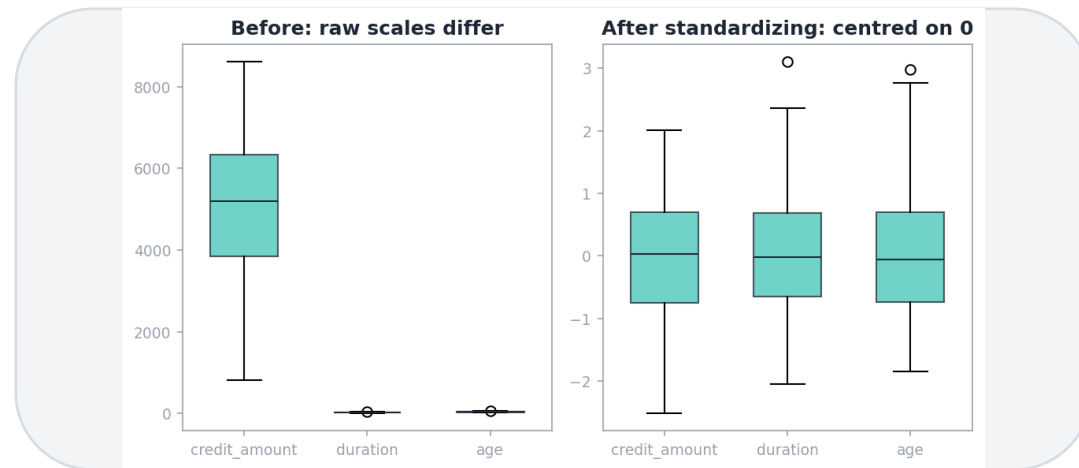
- Rescales numeric attributes to a 0 to 1 range.
- Makes model training less sensitive to feature scale.

SQL (databases)

- A database design technique that reduces redundancy.
- Splits larger tables into smaller linked tables.

Numeric Data: Standardization (Z-score)

$$Z = \frac{X - \mu}{\sigma}$$



1

Rescales to mean 0 and standard deviation 1.

2

Each value becomes 'how many SDs from the average'.

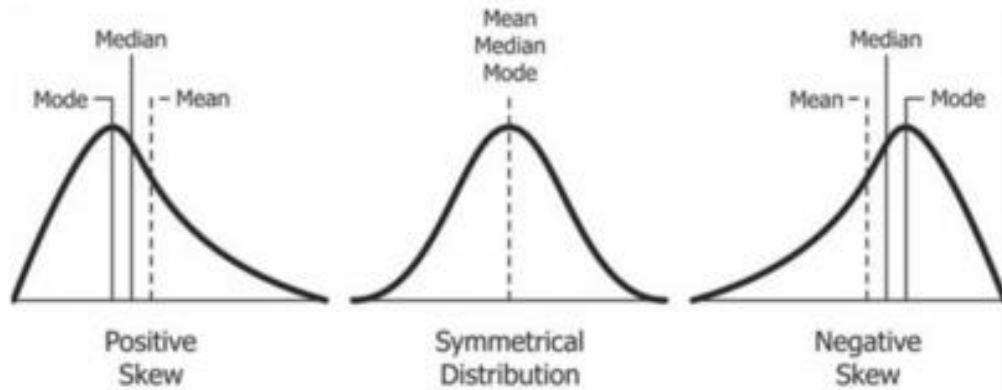
3

The default for linear/logistic regression, SVM and PCA.

4

Less affected by outliers than Min-Max.

Distribution Shape and Skew



[Source: Wikipedia - Skewness](#)

1

Skew describes how lopsided a distribution is.

2

Positive skew: a long right tail; negative skew: a long left tail.

3

Standardization suits roughly symmetric (Gaussian) data.

4

Heavy skew or outliers can call for a transform first (Session 4).

When to Use Normalization vs Standardization

Normalization (Min-Max)

- When the distribution is unknown or not Gaussian.
- When a method needs bounded 0 to 1 inputs.

Standardization (Z-score)

- When the data is roughly Gaussian (bell curve).
- The safe default for most models.

Data Pre-processing



Feature Engineering

Feature Selection and Feature Engineering

Feature selection

- Each column in a DataFrame is a feature.
- Choose the most relevant ones.
- Fewer, better features improve the model.

Feature engineering

- Create new features from existing ones.
- Often matters more than the choice of model.
- Example: instalment-to-income ratio.

Feature Engineering in Practice



Date parts

Extract year/month/day, or a tenure (days since signup).



Binning

`pd.cut` groups a numeric column into bands (risk by age band).



Ratios & interactions

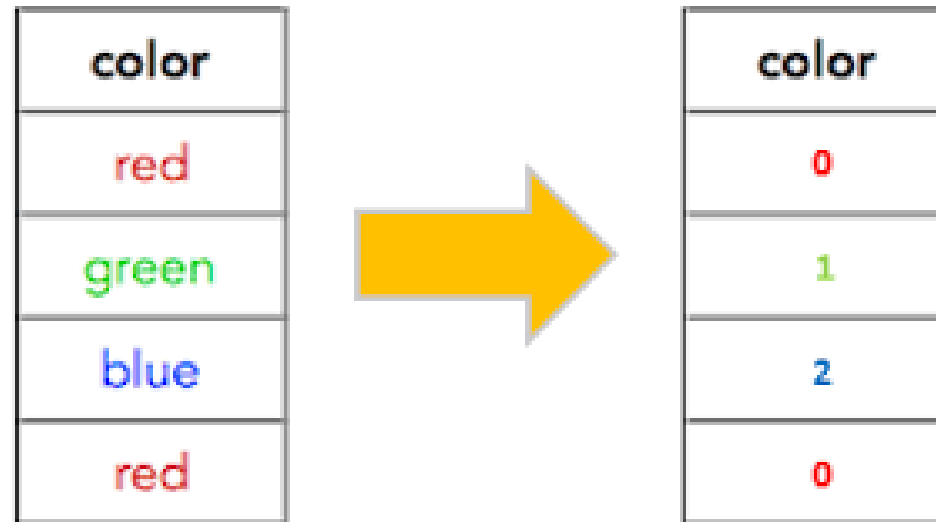
Amount / duration estimates the monthly instalment.

Data Pre-processing



Encoding Categorical Variables

Categorical Data: Label Encoding



Maps each category to an integer.

Compact, but it invents an order.

Fine for ordinal data or tree models.

Misleads models that read the numbers as magnitudes.

[Source: GeeksforGeeks - Label encoding of datasets](#)

Categorical Data: One-Hot Encoding

- Creates a 0/1 column for each category.
- No false ordering between categories.
- The safe default for nominal (unordered) data.
- Cost: more columns when there are many categories.



Source: [DataCamp - One-hot encoding pitfalls](#)

Data Pre-processing

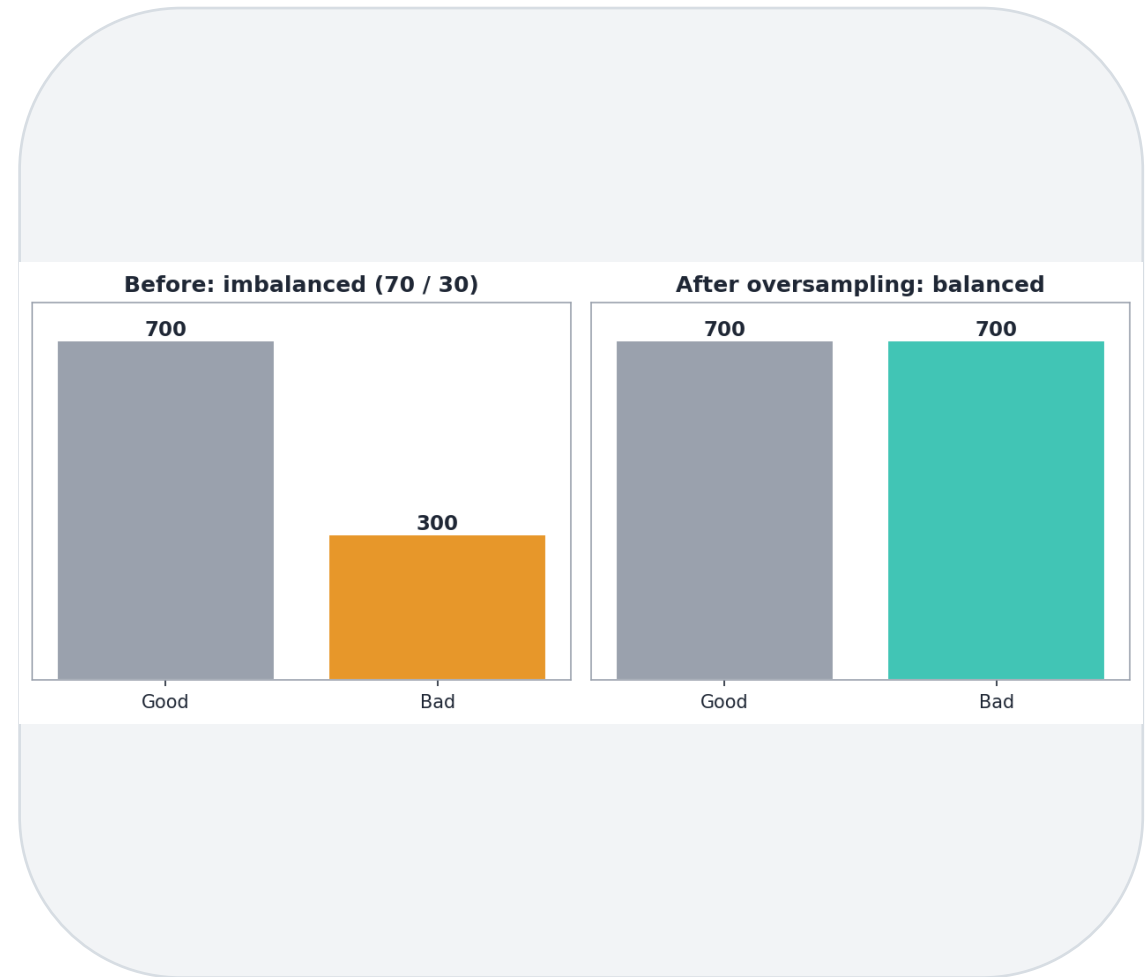


Doing It Safely

Handling Imbalanced Data

- A target is imbalanced when one class is much rarer.
- A model can score everything 'majority' and still look accurate.
- Oversample the minority (or undersample the majority).
- Rebalance the TRAINING data only, never the test set.

[Source: imbalanced-learn](#)



Splitting Without Leakage

1

Split first

- Divide into train and test before any preprocessing.
- Use stratify=y to keep class ratios.

2

Fit on train only

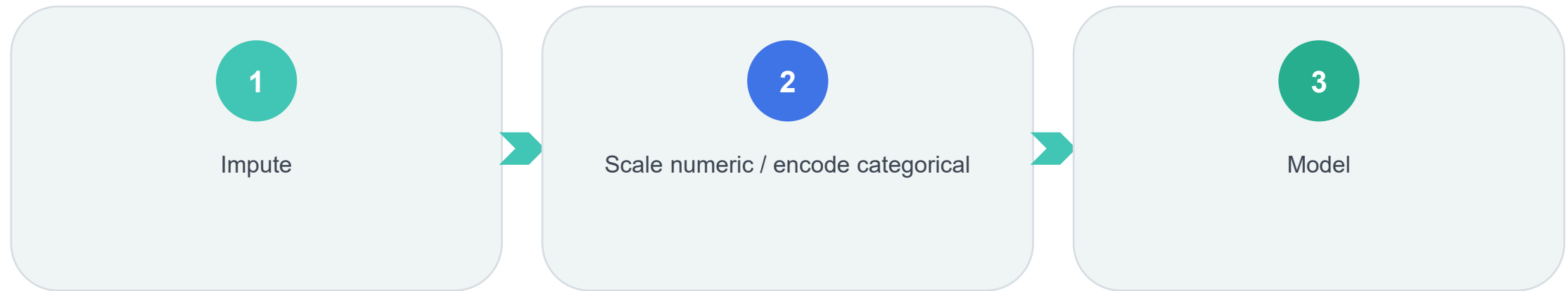
- Fit every scaler/encoder/imputer on the training set alone.

3

Apply to test

- Only transform the test set - never fit on it.
- This keeps validation honest.

Putting It Together: a Pipeline



A Pipeline + ColumnTransformer chains these steps into one object: it guarantees no leakage, stays identical in production, and can be cross-validated as a unit.

In this lesson, we explored



Cleaning & scaling

Imputation, normalization and standardization.



Engineering & encoding

New features, label and one-hot encoding.



Doing it safely

Imbalance, no leakage, and a Pipeline.

Rockborne

Time to get hands-on in Python

We move from theory to practice with a guided walkthrough you run yourself in Python.



Pre-processing - Guided Walkthrough & Case Study



Then: Introduction to Regression

Digging Deeper?



Preprocessing data

scikit-learn



Pipelines &
ColumnTransformer

scikit-learn



imbalanced-learn

over/under-sampling & SMOTE

Questions?